

Indian Sign Language (ISL) Recognition using GCN-LSTM and BiLSTM

1. Project Overview

Indian Sign Language (ISL) recognition is a **spatio-temporal sequence modeling problem** where gestures are represented as sequences of human body and hand movements over time. The objective is to map video sequences to discrete semantic labels (e.g., greetings like Hello, Good Morning, etc.).

This project leverages the dataset:

“Indian Sign Language Greetings Dataset - Sub Variant of INCLUDE”

Each sample consists of an .mp4 video containing a gesture.

Goal

To design a robust pipeline that:

1. Converts raw videos → structured skeletal representations
2. Learns temporal patterns of gestures
3. Classifies gestures using deep learning models

High-Level Pipeline

Video → Frame Augmentation → MediaPipe Landmarks → Sequence Modeling (BiLSTM / GCN-LSTM) → Classification

2. Data Preprocessing Pipeline

2.1 Video Loading

Videos are loaded frame-by-frame using OpenCV:

```
def load_video(video_path):  
    cap = cv2.VideoCapture(video_path)  
    frames = []  
  
    while True:
```

```
ret, frame = cap.read()
if not ret:
    break
frames.append(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

cap.release()
return frames
```

2.2 Frame-Level Augmentation — VideoFrameAugmenter

Applied **consistently across all frames of a video** (important for temporal coherence).

Transformations

Augmentation	Description
Horizontal Flip	Simulates left/right variation
Rotation	Small angular perturbation
Brightness Scaling	Lighting invariance
Zoom	Distance variation
Center Crop	Framing robustness
Temporal Upsampling	Slow motion
Temporal Downsampling	Fast motion

Example Implementation

```
def augment_frame(self, frame, aug_params):
    augmented = frame.copy()

    if aug_params["flip"]:
        augmented = cv2.flip(augmented, 1)

    if aug_params["rotation"] != 0:
        M = cv2.getRotationMatrix2D((w/2, h/2), aug_params["rotation"], 1.0)
        augmented = cv2.warpAffine(augmented, M, (w, h))

    if aug_params["brightness"] != 1.0:
        augmented = cv2.convertScaleAbs(augmented, alpha=aug_params["brightness"])

    return augmented
```

Temporal Augmentation

```
def temporal_upsample(self, frames, factor=1.5):
    return [f.copy() for f in frames for _ in range(int(factor))]

def temporal_downsample(self, frames, factor=1.5):
    return frames[::int(factor)]
```

2.3 MediaPipe Landmark Extraction — MediaPipeLandmarkExtractor

Extracts structured skeletal features using **MediaPipe Holistic**.

Extracted Features per Frame

Component	Count	Features
Left Hand	21	(x, y, z) → 63
Right Hand	21	(x, y, z) → 63
Pose	33	(x, y, z, visibility) → 132

Total Features = 63 + 63 + 132 = 258

Relative Normalization

Key idea: remove global position and scale variance.

```
def relative_normalize(lh, rh, pose):  
    if np.any(rh):  
        origin = rh.reshape(21,3)[0]  
    elif np.any(lh):  
        origin = lh.reshape(21,3)[0]  
    else:  
        origin = np.zeros(3)  
  
    lh = lh.reshape(21,3) - origin  
    rh = rh.reshape(21,3) - origin  
  
    pose = pose.reshape(33,4)  
    pose[:, :3] -= origin  
  
    l_shoulder = pose[11][:3] r_shoulder = pose[12][:3]  
    scale = np.linalg.norm(l_shoulder - r_shoulder) + 1e-6  
  
    lh /= scale rh /=  
    scale pose[:, :3] /=  
    scale  
  
    return np.concatenate([lh.flatten(), rh.flatten(), pose.flatten()])
```

- This is also implemented in the inference

2.4 Landmark-Level Augmentation — LandmarkAugmenter

Gaussian Jitter (Hands Only)

```
seq[:, :hand_dim] += np.random.normal(0, noise_factor, seq[:, :hand_dim].shape)
```

Temporal Stretching

```
indices = np.linspace(0, original_length - 1, new_length).astype(int)
stretched = sequence[indices]
```

2.5 Final Dataset Pipeline

```
X, y = process_dataset(
    dataset_path,
    max_frames=60,
    frame_aug_count=1,
    landmark_aug_count=0
)
```

Key Properties

- Fixed length: 60 frames
- Padding with zeros if shorter
- Output shape: $X \in \mathbb{R}^{(N, 60, 258)}$

3. Approach 1 — Bidirectional LSTM (Baseline)

3.1 Architecture

```
model = Sequential([
    Masking(mask_value=0.0, input_shape=input_shape),

    Bidirectional(LSTM(256, return_sequences=True, dropout=0.2)),
    BatchNormalization(),

    Bidirectional(LSTM(128, return_sequences=False, dropout=0.2)),
```

```
BatchNormalization(),

Dense(256, activation='relu'),
Dropout(0.5),

Dense(128, activation='relu'),
Dropout(0.3),

Dense(num_classes, activation='softmax')
])
```

3.2 Input Representation

Input shape: (60, 258)

Each frame is treated as a flat feature vector

3.3 Training Setup

- Optimizer: **Adam (lr = 0.001)**
- Loss: **Categorical Crossentropy**
- Callbacks:
 - EarlyStopping (patience=20)
 - ReduceLROnPlateau ModelCheckpoint
 - Class balancing

```
compute_class_weight(...)
```

3.4 Why It Works

- LSTM captures **temporal dependencies**
- Bidirectional flow improves context understanding

Limitations

- Ignores **spatial structure of joints**
- Treats all 258 features equally (no relational bias)
- Cannot model joint connectivity

4. Approach 2 — Graph Convoluted Network + BiLSTM (Primary Model)

4.1 Graph Construction

Function:

```
def get_adjacency_matrix():
```

Graph Definition

- Total joints: **75**
- 21 (LH) + 21 (RH) + 33 (Pose)

Includes:

- Hand skeleton edges
- Pose skeleton edges
- Cross-body wrist connections
- Self-loops

Normalization

Normalized Adjacency Matrix:

$$A_{\text{norm}} = D^{-1/2} * A * D^{-1/2}$$

Where:

$$D_{ii} = \sum_j (A_{ij})$$

4.2 GCN Layer

Mathematical Formulation

$$H' = \text{activation}(A_{\text{norm}} * H * W + b)$$

Implementation

```
class GCNLayer(tf.keras.layers.Layer):
    def call(self, x, A):
        support = tf.matmul(x, self.W) + self.b
        output = tf.matmul(A, support)
        return self.activation_fn(output)
```

TimeDistributed Application

```
TimeDistributed(GCNStep(...))
```

→ Applies GCN **per frame**

4.3 Landmark Reshaping

```
def reshape_landmarks_for_gcn(X):
```

Transformation:

Input shape: (60, 258)

Reshaped to: (60, 75, 4)

Joint Type	Features
Hands	(x, y, z, 0)
Pose	(x, y, z, visibility)

4.4 Full Model Architecture

Input (60, 300)

```
→ Reshape (60, 75, 4)
→ TimeDistributed(GCN 64)
→ TimeDistributed(GCN 128)
→ TimeDistributed(Flatten)
→ Masking
→ BiLSTM(256)
→ BatchNorm
→ BiLSTM(128)
→ BatchNorm
```

→ Dense → Dropout
→ Dense → Dropout
→ Softmax

Code Snippet

```
x = TimeDistributed(GCNStep(64, A))(x)
x = TimeDistributed(GCNStep(128, A))(x)
x = TimeDistributed(Flatten())(x)

x = Bidirectional(LSTM(256, return_sequences=True))(x)
x = Bidirectional(LSTM(128))(x)
```

■ Why GCN is Better

Aspect	Benefit
Spatial modeling	Captures joint relationships
Structural bias	Mimics human skeleton
Robustness	Better generalization

5. Comparison of Approaches

Feature	BiLSTM	GCN-LSTM
Input	Flat (258)	Structured (75×4)
Spatial Awareness	■ None	■ Yes ■ Yes
Temporal Modeling	■ Yes	Very Strong
Inductive Bias	Weak	
Generalization	Moderate	

6. Training & Evaluation Strategy

Data Split

```
train_test_split(..., stratify=y)
```

- 80% Train
- 20% Test

Training Enhancements

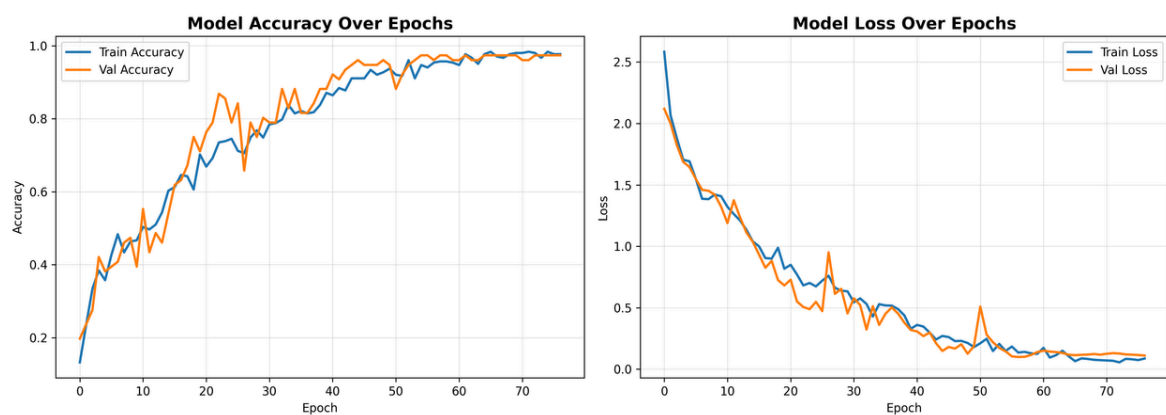
Class weights → handle imbalance

Early stopping → avoid overfitting

LR scheduling → adaptive learning

Metrics

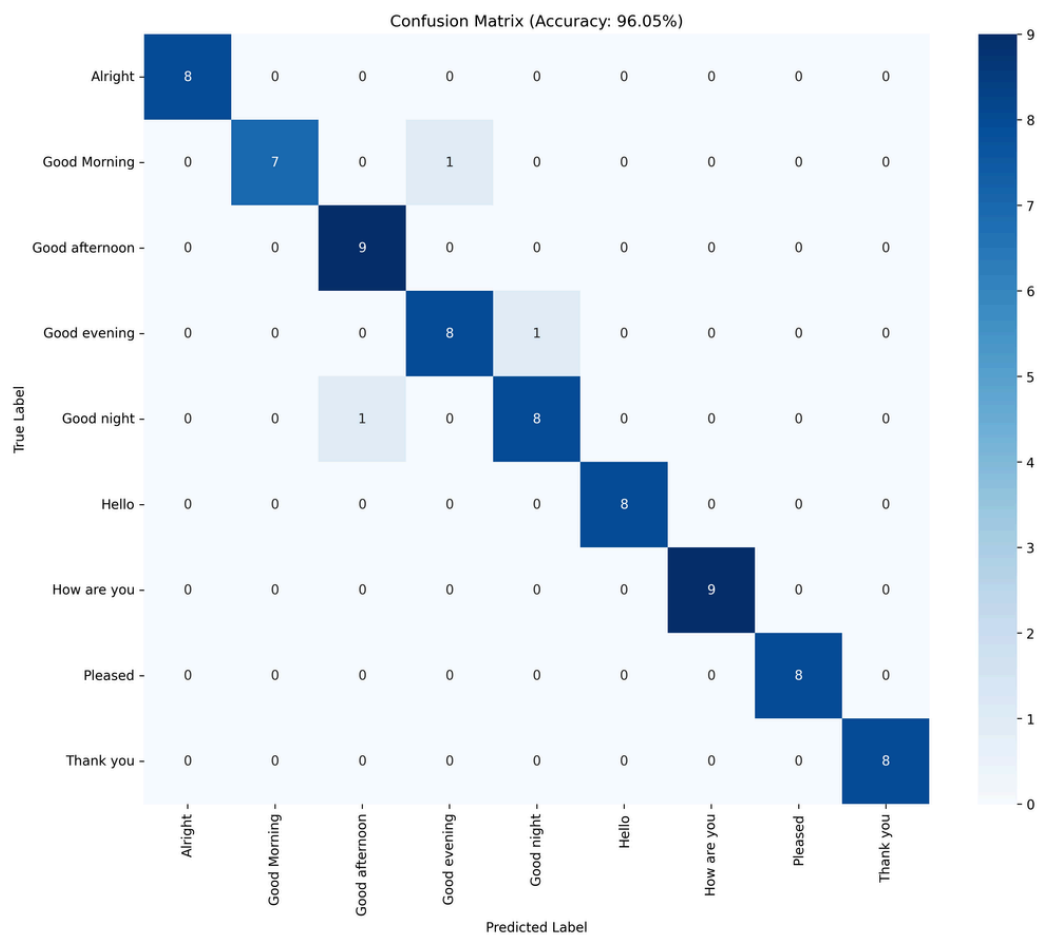
- Accuracy & Loss Curves



- Classification Report

Classification Report:				
	precision	recall	f1-score	support
Alright	1.000	1.000	1.000	8
Good Morning	1.000	0.875	0.933	8
Good afternoon	0.900	1.000	0.947	9
Good evening	0.889	0.889	0.889	9
Good night	0.889	0.889	0.889	9
Hello	1.000	1.000	1.000	8
How are you	1.000	1.000	1.000	9
Pleased	1.000	1.000	1.000	8
Thank you	1.000	1.000	1.000	8
accuracy			0.961	76
macro avg	0.964	0.961	0.962	76
weighted avg	0.962	0.961	0.960	76

- Confusion Matrix



7. Inference Pipeline

Function:

```
predict_from_video(...)
```

Steps

- Load class names
- Rebuild model architecture
- Load weights
- Extract landmarks
- Reshape for GCN
- Predict

Prediction Logic

```
probs = model.predict(landmark_input)[0]  
pred_idx = np.argmax(probs)
```

Dummy Predictions

Top 3 Predictions:

1. Good night: 0.717
2. Thank you: 0.130
3. Alright: 0.127

Final Prediction: Good night (confidence: 0.717)